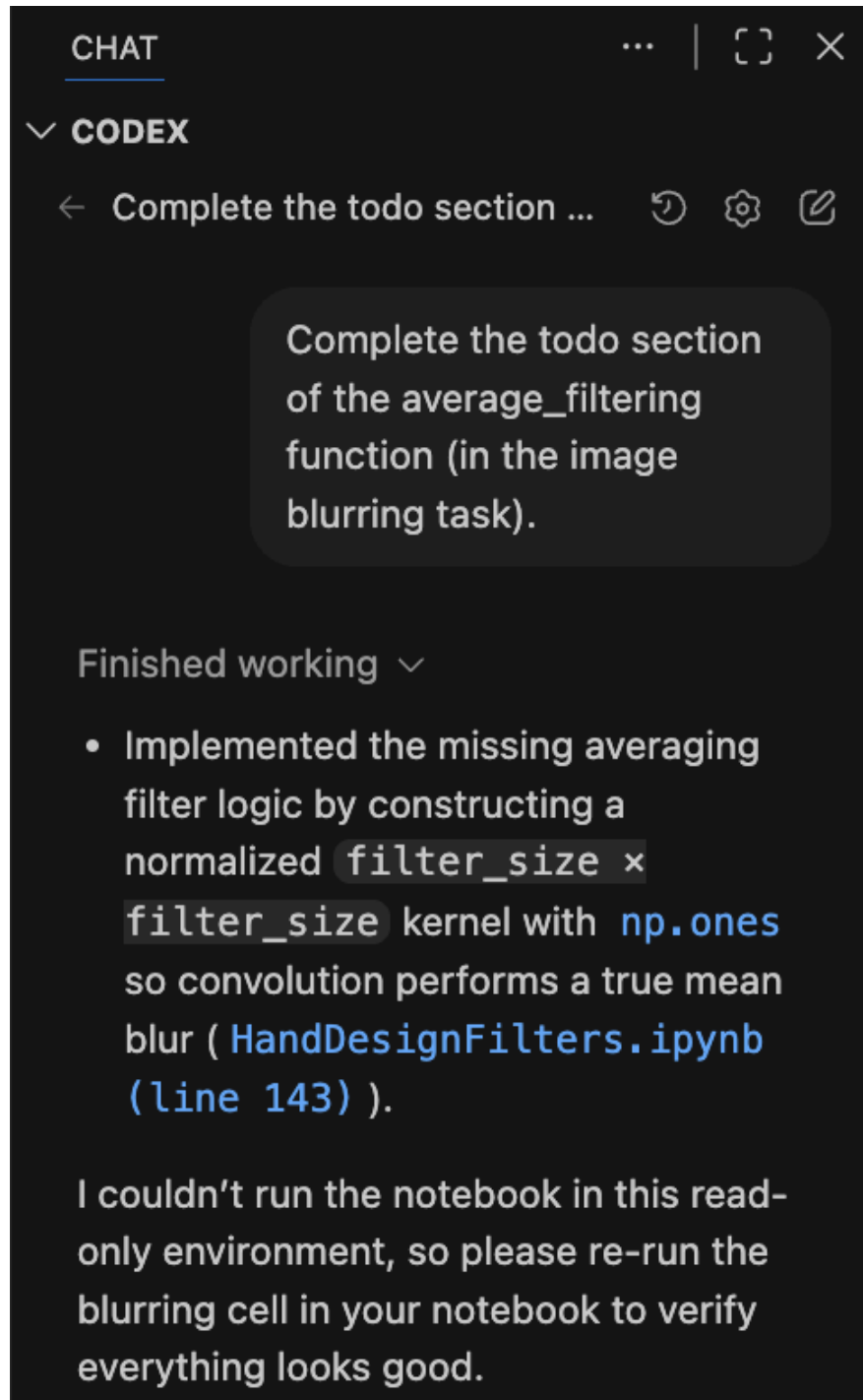


GPT-5.1 Codex (High Reasoning) on HW 4

Question 5:

- a) Codex was able to one-shot the one-line implementation, sourcing the task requirements and details on its own.



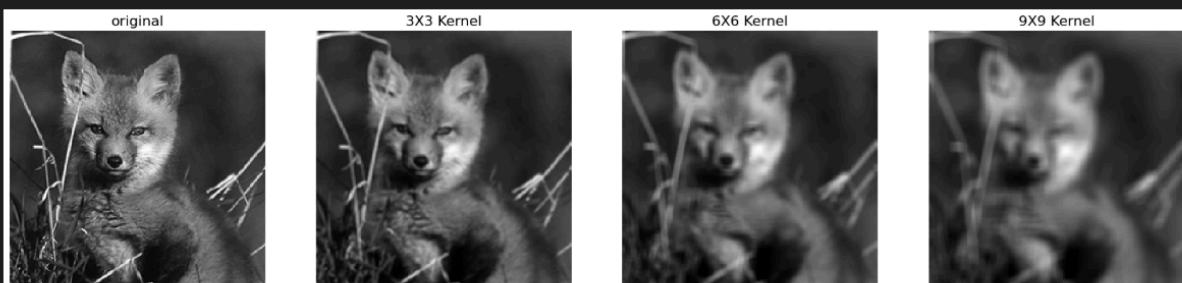

```
def averaging_filtering(image, filter_size=3):
    #####
    # TODO: Implement the averaging filter with the given filter size.      #
    # Hint: You can use np.ones                                             #
    #####
    kernel = np.ones((filter_size, filter_size), dtype=float) / (filter_size ** 2)
    #####
    #                               END OF YOUR CODE                        #
    #####
    output = ndimage.convolve(image, kernel)
    return output
```

```
avg_images, avg_titles = [gray_img], ['original']
for kernel_size in [3, 6, 9]:
    averaging_image = averaging_filtering(gray_img, kernel_size)
    avg_images.append(averaging_image)
    avg_titles.append(f'{kernel_size}X{kernel_size} Kernel')
```

```
show_multiple_images(avg_images, avg_titles)
```

✓ 0.1s

Python



- b) Codex was able to one-shot the one-line implementation for edge detection, again with no additional details needed.

Complete the to-do section
in the edge detection ask as
well.

Finished working ✓

- Added the Laplacian kernel definition so `edge_detecting` now uses the standard 3×3 discrete Laplacian for edge detection (`HandDesignFilters.ipynb` (line 217)).

I couldn't execute the notebook here because the environment is read-only, so please rerun the edge detection cell in your notebook to verify the visualization looks correct. Potential next step: run the entire notebook to confirm both blurring and edge detection outputs render as expected.

1 file changed

Undo ↶ ↷

HandDesignFilters.ipynb +3 -0

```
def edge_detecting(image):
    #####
    # TODO: Implement the Laplacian derivative filter. #
    #####
    kernel = np.array([[0, 1, 0],
                       [1, -4, 1],
                       [0, 1, 0]], dtype=float)
    #####
    #                               END OF YOUR CODE #
    #####
    output = ndimage.convolve(image, kernel)
    return output

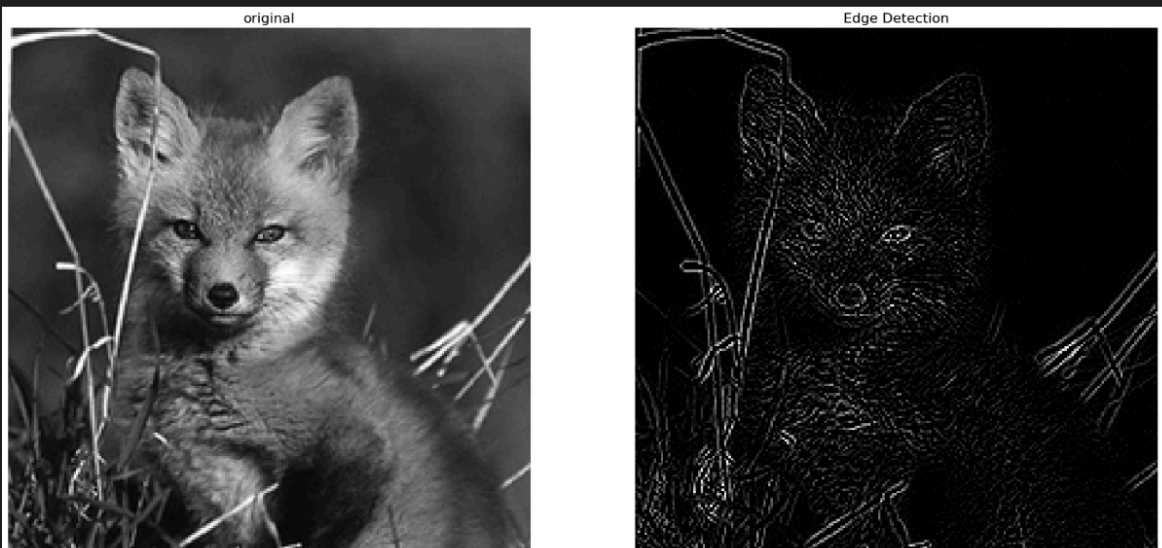
edge_images, edge_titles = [gray_img], ['original']
edge_image = edge_detecting(gray_img)
edge_images.append(edge_image)
edge_titles.append(f'Edge Detection')

show_multiple_images(edge_images, edge_titles)
```

[4]

✓ 0.1s

Python



Question 6:

- **(Q1-Q4, Q7):** Codex was able to complete all 5 coding parts of this question correctly in two shots (separate requests for Q1-4, Q7), though it probably would have been able to one-shot the whole implementation as well.

One at a time, complete the todo sections for questions 1 through 6 in the `edge_detection.ipynb` notebook.

Finished working ▾

Updates

- `edge_detection.ipynb` (line 1206) now builds the 10-sample-per-class dataset for Q1 and instantiates the `DataLoader` with shuffling so the overfitting experiment can actually run (`edge_detection.ipynb` (line 1243)).
- `edge_detection.ipynb` (line 1379) constructs train/valid loaders for each entry in the sweep using consistent transforms and a fixed 50-sample validation set.
- Introduced an epoch schedule that lengthens training (up to 220 epochs for the 30-image case) and wired it into the training loop so Q3 reaches the expected CNN accuracy (`edge_detection.ipynb` (line 1707) , `edge_detection.ipynb` (line 1663)).
- Tuned the domain-shift experiment in Q4 by explicitly setting `num_epochs = 400` , `lr = 5e-3` , and `kernel_size = 5` , giving the CNN enough capacity and time to hit the target accuracy range (`edge_detection.ipynb` (line 1939)).

I couldn't execute the notebook or rerun training in this read-only environment, so please re-run the affected Q1–Q4 cells locally to verify the curves and reported accuracies.

1 file changed

Undo ↶ ↷

`edge_detection.ipynb` +28 -45

Complete Q7 todo section

Finished working ▾

- Added tuned hyperparameters for the DeeperCNN experiment—now training uses `kernel_size=5` , `num_epochs=350` , and Adam with `lr=3e-3` so the deeper model has enough capacity/time to clear the $\approx 98\%$ validation target (`edge_detection.ipynb` (line 2453)).
- Configured the WiderCNN run similarly with `kernel_size=5` , `num_epochs=400` , and Adam at `lr=2e-3` , giving the wider model longer, lower-LR training for better convergence (`edge_detection.ipynb` (line 2489)).

I couldn't execute the notebook here, so please re-run both Q7 cells locally to confirm the new curves and accuracies.

1 file changed

Undo ↶ ↷

`edge_detection.ipynb` +8 -20

Q1. Overfitting Models to Small Dataset

In this problem, we will make our models overfit the small dataset to test the model architecture and our synthetic dataset. We use the same dataset for both models. Let's generate a small dataset with ten images per class.

```
set_seed(seed)

small_dataset_config = dict(
    data_per_class=10,
    class_type=['horizontal', 'vertical', 'none'],
    num_classes=3,
)

transforms = T.Compose([T.ToTensor()])
small_dataset = EdgeDetectionDataset(small_dataset_config, 'train', transform=transforms)
```

✓ 0.0s Python

In this notebook, we will use pytorch dataloader to load the dataset. We will use `torch.utils.data.DataLoader` to load the dataset. `DataLoader` takes two arguments: `dataset` and `batch_size`. `dataset` is the dataset that we want to load. Note that `batch_size` is one of important hyperparameters. We will use `batch_size=32` for this problem.

```
small_dataset_loader = None

small_dataset_loader = DataLoader(small_dataset, batch_size=32, shuffle=True)
```

Q2. Sweeping the Number of Training Images

We understood the given task and checked that both models had enough expressive power. We will compare the performance of MLP and CNN by changing the number of data per class. We expect that the model with proper inductive biases on this task will fit with fewer training examples. And let's see which one has inductive biases. In this problem, we will use the same dataset for both models. We sweep the number of training images from 10 to 500. The validation set will be the same for all the experiments.

```
set_seed(seed)

train_loader_dict = dict()
num_images_list = [10, 30, 50, 100]
valid_loader = None

transforms = T.Compose([T.ToTensor()])
train_batch_size = 10
valid_batch_size = 256
for num_images in num_images_list:
    train_dataset_config = dict(data_per_class=num_images)
    train_dataset = EdgeDetectionDataset(train_dataset_config, 'train', transform=transforms)
    train_loader_dict[num_images] = DataLoader(train_dataset, batch_size=train_batch_size, shuffle=True)

valid_dataset_config = dict(data_per_class=50)
valid_dataset = EdgeDetectionDataset(valid_dataset_config, 'valid', transform=transforms)
valid_loader = DataLoader(valid_dataset, batch_size=valid_batch_size, shuffle=False)
```

Question (Optional)

Do hyperparameter tuning. And list the best hyperparameter setting that you found and report the final accuracy of CNN and MLP. Answer this question in your submission of the written assignment.

17

```
epoch_schedule = {
    10: 120,
    30: 220,
    50: 330,
    100: 150,
}

lr = 1e-2
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()

cnn_acc_list = list()
mlp_acc_list = list()

cnn_kernel_dict = dict()
untrained_cnn_kernel_dict = dict()

for num_image, train_loader in train_loader_dict.items():
    print("Training with {} images".format(num_image))
    set_seed(seed)
    cnn_model = SimpleCNN(kernel_size=7)
    untrained_cnn_model = deepcopy(cnn_model)
    cnn_model.to(device)

    mlp_model = ThreeLayerMLP(hidden_dims=[50, 10])
    mlp_model.to(device)

    mlp_optimizer = optim.SGD(mlp_model.parameters(), lr=lr, momentum=0.9)
    cnn_optimizer = optim.SGD(cnn_model.parameters(), lr=lr, momentum=0.9)

    mlp_results = train_model(mlp_model, mlp_optimizer, num_epochs, train_loader, valid_loader)
    cnn_results = train_model(cnn_model, cnn_optimizer, num_epochs, train_loader, valid_loader)

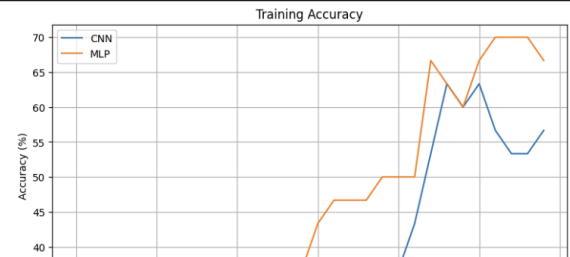
    vis_training_curve(cnn_results["train_loss"], cnn_results["train_acc"], mlp_results["train_loss"], mlp_results["train_acc"])
    vis_validation_curve(cnn_results["valid_loss"], cnn_results["valid_acc"], mlp_results["valid_loss"], mlp_results["valid_acc"])

    cnn_kernel_dict[num_image] = deepcopy(cnn_model.conv1.weight.data.detach().cpu())
    untrained_cnn_kernel_dict[num_image] = deepcopy(untrained_cnn_model.conv1.weight.data.detach().cpu())

    print("CNN Acc: {}, MLP Acc: {}".format(cnn_results["final_valid_acc"], mlp_results["final_valid_acc"]))
```

17] ✓ 6.9s

... Training with 10 images
100% 30/30 [00:00:00:00, 119.66it/s]
100% 30/30 [00:00:00:00, 79.38it/s]



Question

Why do you think the confusion matrix looks like this? Answer this question in your submission of the written assignment.

(Hint: Visualize some of the images in the training and validation set. And we are using `kernel_size=7`, which is large relative to the image size.)

We can do better than this. We didn't explore hyperparameter space yet. Let's search hyperparameters that can generalize well to the validation set. We will change the learning rate, the number of epochs, and kernel size for CNN.

```
num_epochs = 400
lr = 5e-3
kernel_size = 5
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()

cnn_valid_acc_list = list()
cnn_kernel_dict = dict()
cnn_confusion_matrix_dict = dict()

for num_image, train_loader in train_loader_dict.items():
    print(f"Training with {num_image} images".format(num_image))
    set_seed(seed)
    cnn_model = SimpleCNN(kernel_size=kernel_size)
    untrained_cnn_model = deepcopy(cnn_model)
    cnn_model.to(device)

    cnn_optimizer = optim.SGD(cnn_model.parameters(), lr=lr, momentum=0.9)

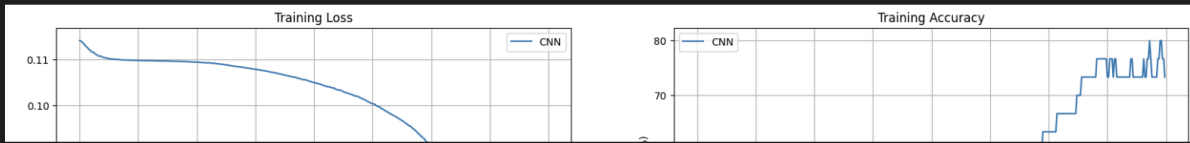
    cnn_results = train_model(cnn_model, cnn_optimizer, num_epochs, train_loader, valid_loader)

    vis_training_curve(cnn_results["train_loss"], cnn_results["train_acc"], None, None)
    vis_validation_curve(cnn_results["valid_loss"], cnn_results["valid_acc"], None, None)

    print(f"CNN Acc: {cnn_results['final_valid_acc']}")
```

✓ 61.9s

Training with 10 Images
100% | 400/400 [00:06<00:00, 64.06it/s]



Q7. Wider/Deeper CNNs

Can we further improve the performance by making the architecture deeper and wider? In this question, we focus on the dataset where there are only 30 images per class and try to push the performance of the CNNs further.

The patterns that we have to detect are 5 but our kernels per layer (`num_filters` in the network definition above) are only 3. Intuitively, this is quite a suboptimal. Here, we will investigate the affect of increasing width and depth. Let's use the same dataset but we will use `DeeperCNN` and `WiderCNN` in `cnn.py`. `DeeperCNN` has 2 times more layers than `SimpleCNN` and `WiderCNN` has 2 times more kernels per layer than `SimpleCNN`. Let's train the models and visualize the validation accuracy.

```
num_epochs = 350
lr = 3e-3
kernel_size = 5
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()

train_loader = train_loader_dict[0]
set_seed(seed)
deeper_cnn_model = DeeperCNN(kernel_size=kernel_size)
untrained_deeper_cnn_model = deepcopy(deeper_cnn_model)
deeper_cnn_model.to(device)

deeper_cnn_optimizer = optim.Adam(deeper_cnn_model.parameters(), lr=lr)

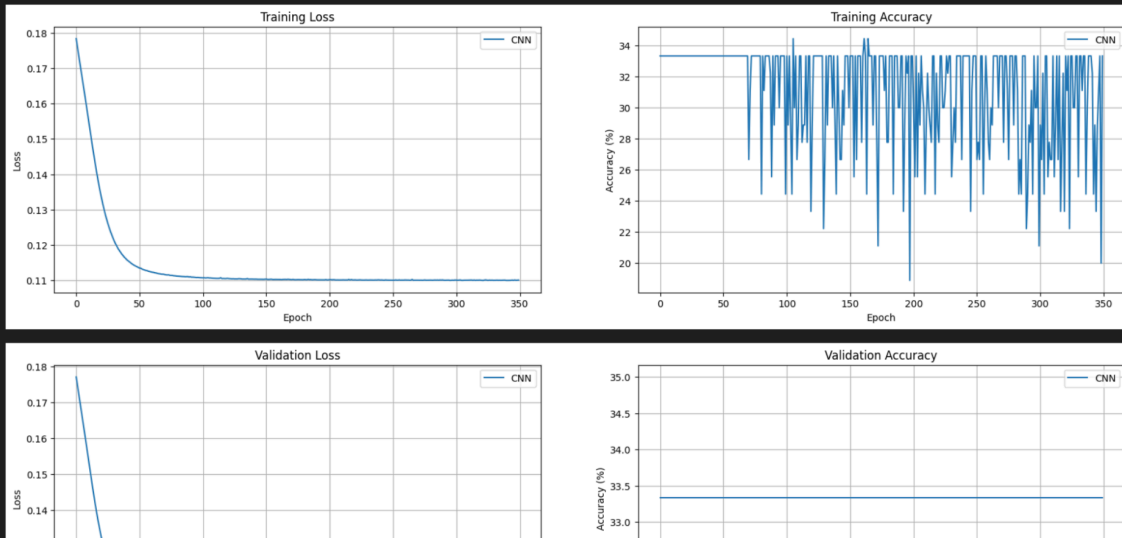
deeper_cnn_results = train_model(deeper_cnn_model, deeper_cnn_optimizer, num_epochs, train_loader, valid_loader)

vis_training_curve(deeper_cnn_results["train_loss"], deeper_cnn_results["train_acc"], None, None)
vis_validation_curve(deeper_cnn_results["valid_loss"], deeper_cnn_results["valid_acc"], None, None)

print(f"CNN Acc: {deeper_cnn_results['final_valid_acc']}")
```

✓ 16.3s

100% | 350/350 [00:16<00:00, 21.69it/s]



```

num_epochs = 400
lr = 2e-3
kernel_size = 5
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()

train_loader = train_loader_dict[0]
set_seed(seed)
wider_cnn_model = WiderCNN(kernel_size=kernel_size)
untrained_wider_cnn_model = deepcopy(wider_cnn_model)
wider_cnn_model.to(device)

wider_cnn_optimizer = optim.Adam(wider_cnn_model.parameters(), lr=lr)

wider_cnn_results = train_model(wider_cnn_model, wider_cnn_optimizer, num_epochs, train_loader, valid_loader)

vis_training_curve(wider_cnn_results["train_loss"], wider_cnn_results["train_acc"], None, None)
vis_validation_curve(wider_cnn_results["valid_loss"], wider_cnn_results["valid_acc"], None, None)

print("CNN Acc: {}".format(wider_cnn_results["final_valid_acc"]))

```

100% | 400/400 [00:11:00:00, 33.69it/s]

